

DA6401W - Assignment 3

Department of Data Science and AI, IIT Madras

Mar 2026

Topics covered: Initialization, adagrad, rmsprop, adam, Newton's method, conjugate gradient

Submit your solutions in one PDF file with all answers, codes, plots and outputs written/printed in that file.

1. **Explore what causes slow training in multilayer perceptron neural networks even in the absence of vanishing gradients (a statistical view).**

To observe, visualize, and reason about unexpected change of input-distribution in hidden layers during neural network training. Use pytorch's module and pytorch's forward hooks for implementation.

Problem Setup

- Dataset: MNIST or Fashion-MNIST (you can import dataset from torchvision.datasets.MNIST)
- Input shape: 28×28
- 28×28 grayscale images, flattened to 784
- Batch size: fixed (e.g., 64)

Task

- Train a multi-layer perceptron (MLP) for classification.
- During training, record and visualize the evolution of hidden-layer activation distributions.

Network Architecture

Use a simple, deliberately unstable model, below is an example

Sequential Pytorch Model:

Input $\rightarrow$ Linear($784 \rightarrow 256$) $\rightarrow$ ReLU

Linear($256 \rightarrow 128$) $\rightarrow$ ReLU

Linear($128 \rightarrow 64$) $\rightarrow$ ReLU

Linear($64 \rightarrow 10$)

What exactly to record

Choose one fixed mini-batch from the training set.

At the end of each training epoch / fixed number of gradient update steps:

Run a forward pass on this fixed batch

Using Pytorch's forward hooks, record activations (output of activation layer) from each hidden layer, of shape (batch_size, num_channel):

Visualization Requirements:

Histogram Evolution At the end of each training epoch, flatten the activation tensor of shape (B, C) into a one-dimensional vector of size $B \times C$ and plot its histogram.

The histogram should be updated after every epoch. This can be done by either:

- saving one histogram figure per epoch, or
- creating an animation that shows the evolution of the activation distribution over time.

Mean and Variance Over Time

For each hidden layer and at every epoch, compute:

- the mean of the activations, and
- the variance of the activations.

Plot the mean versus training epoch and the variance versus training epoch. Each hidden layer should have its own set of plots.

2. **Gradient Accumulation and Memory Constraints in Deep Learning** Training modern neural networks often requires large batch sizes for stable gradients and faster convergence. However, GPU memory limits often make large batches infeasible. In this problem, we will analyze optimizer memory usage and derive why *gradient accumulation* is used in frameworks such as PyTorch.

Assume:

- A neural network with P parameters
- Each parameter stored in 32-bit floating point (4 bytes)
- A batch size of B
- Gradient accumulation steps or number of mini-batches K .

Memory Footprint of Optimizers

Consider the memory required to store parameters, gradients, and optimizer states.

- (a) For **Stochastic Gradient Descent (SGD)** without momentum, determine the total memory required for storing parameters and gradients.

- (b) For **SGD with Momentum**, the optimizer additionally stores a momentum vector of the same size as the parameters. Compute the total memory requirement.
- (c) For the **Adam optimizer**, two additional vectors m_t and v_t are stored for each parameter. Compute the total optimizer memory footprint.
- (d) Suppose $P = 10^9$ parameters. Compute the memory required (in GB) for each of the above optimizers.

Batch Size and Activation Memory

During training, memory is also required to store intermediate activations for backpropagation.

Assume the activation memory per sample is A bytes.

- (a) Express the total activation memory required for batch size B .
- (b) Suppose the GPU memory limit is M bytes. Write an inequality involving B , A , and optimizer memory that must hold for training to fit into memory.
- (c) Explain why increasing batch size B may become infeasible for very large models.

Convergence vs Batch Size

Empirical studies suggest that very small batch sizes produce noisy gradient estimates and slow convergence.

- (a) Suppose we desire an effective batch size B_{large} , but due to memory limits we can only fit a mini-batch of size b where $b < B_{\text{large}}$.
- (b) Propose a strategy that allows us to simulate a larger batch using multiple smaller mini-batches without increasing memory usage.
- (c) Let K denote the number of such mini-batches processed before updating the parameters. Express the relationship between B_{large} , b , and K .

Equivalence of Gradient Accumulation and Large Batch Updates

Let the loss over a batch be defined as the average loss:

$$L(\theta) = \frac{1}{B} \sum_{i=1}^B \ell(x_i, \theta)$$

where $\ell(x_i, \theta)$ is the loss for sample i .

Assume we divide the batch into K mini-batches each of size $b = B/K$.

- (a) Write the gradient of the full batch loss $\nabla_{\theta} L(\theta)$.

(b) Let g_k be the gradient computed from mini-batch k :

$$g_k = \frac{1}{b} \sum_{i \in \mathcal{B}_k} \nabla_{\theta} \ell(x_i, \theta)$$

Show that

$$\nabla_{\theta} L(\theta) = \frac{1}{K} \sum_{k=1}^K g_k$$

(c) Explain why gradient accumulation allows training with large effective batch sizes even when GPU memory cannot hold the entire batch simultaneously.

Solution: Given: P parameters, FP32 (4 bytes each). Batch size B . Activation memory per sample A bytes. Gradient accumulation steps K .

1. Memory Footprint of Optimizers

(a) **SGD (no momentum).** We store:

- parameters θ : P floats
- gradients $\nabla \theta$: P floats

So total memory

$$M_{\text{SGD}} = (P + P) \cdot 4 = 2P \cdot 4 = 8P \text{ bytes.}$$

(b) **SGD + Momentum.** Additionally store momentum buffer v : P floats.

$$M_{\text{Mom}} = (P + P + P) \cdot 4 = 3P \cdot 4 = 12P \text{ bytes.}$$

(c) **Adam.** Additionally store first and second moments (m_t, v_t) : $2P$ floats.

$$M_{\text{Adam}} = (P + P + P + P) \cdot 4 = 4P \cdot 4 = 16P \text{ bytes.}$$

(Parameters + gradients + m_t + v_t .)

(d) **Numerical:** $P = 10^9$. Compute in bytes then convert to GB (using 1 GB = 10^9 bytes for simplicity).

$$M_{\text{SGD}} = 8 \cdot 10^9 \approx 8 \text{ GB}$$

$$M_{\text{Mom}} = 12 \cdot 10^9 \approx 12 \text{ GB}$$

$$M_{\text{Adam}} = 16 \cdot 10^9 \approx 16 \text{ GB.}$$

2. Batch Size and Activation Memory

- (a) If activation memory per sample is A bytes, then for batch size B :

$$M_{\text{act}}(B) = B \cdot A.$$

- (b) Let GPU memory limit be M bytes. Training fits if:

$$M_{\text{opt}} + M_{\text{act}}(B) \leq M,$$

i.e.

$$M_{\text{opt}} + BA \leq M,$$

where M_{opt} is one of M_{SGD} , M_{Mom} , M_{Adam} plus any other fixed overhead.

- (c) As B increases, activation memory BA grows linearly in B . For very large models, M_{opt} is already huge, leaving limited headroom for activations; thus even moderate increases in B can violate

$$M_{\text{opt}} + BA \leq M.$$

Hence large batches become infeasible due to activation memory (and other runtime buffers) even if parameters fit.

3. Convergence vs Batch Size

- (a) If we want effective batch size B_{large} but only fit b ($b < B_{\text{large}}$), then we cannot compute the gradient on all B_{large} samples in one forward/backward pass.
- (b) Strategy: **gradient accumulation**. Process K mini-batches sequentially, accumulate their gradients in a buffer, and perform *one* optimizer step after K mini-batches.
- (c) If each mini-batch has size b and we accumulate for K steps, the effective batch size is

$$B_{\text{large}} = Kb \iff K = \frac{B_{\text{large}}}{b}.$$

4. Equivalence of Gradient Accumulation and Large Batch Updates

Let the full-batch (average) loss be

$$L(\theta) = \frac{1}{B} \sum_{i=1}^B \ell(x_i, \theta).$$

Partition the B samples into K disjoint mini-batches $\mathcal{B}_1, \dots, \mathcal{B}_K$ each of size $b = B/K$.

(a) The full-batch gradient is

$$\nabla_{\theta} L(\theta) = \nabla_{\theta} \left(\frac{1}{B} \sum_{i=1}^B \ell(x_i, \theta) \right) = \frac{1}{B} \sum_{i=1}^B \nabla_{\theta} \ell(x_i, \theta).$$

(b) Define mini-batch gradients

$$g_k = \frac{1}{b} \sum_{i \in \mathcal{B}_k} \nabla_{\theta} \ell(x_i, \theta).$$

Now compute

$$\frac{1}{K} \sum_{k=1}^K g_k = \frac{1}{K} \sum_{k=1}^K \left(\frac{1}{b} \sum_{i \in \mathcal{B}_k} \nabla_{\theta} \ell(x_i, \theta) \right) = \frac{1}{Kb} \sum_{k=1}^K \sum_{i \in \mathcal{B}_k} \nabla_{\theta} \ell(x_i, \theta).$$

Since $\{\mathcal{B}_k\}$ partitions the B examples, the double sum covers each $i \in \{1, \dots, B\}$ exactly once:

$$\frac{1}{Kb} \sum_{k=1}^K \sum_{i \in \mathcal{B}_k} \nabla_{\theta} \ell(x_i, \theta) = \frac{1}{Kb} \sum_{i=1}^B \nabla_{\theta} \ell(x_i, \theta).$$

Using $Kb = B$:

$$\frac{1}{Kb} \sum_{i=1}^B \nabla_{\theta} \ell(x_i, \theta) = \frac{1}{B} \sum_{i=1}^B \nabla_{\theta} \ell(x_i, \theta) = \nabla_{\theta} L(\theta).$$

Thus,

$$\nabla_{\theta} L(\theta) = \frac{1}{K} \sum_{k=1}^K g_k.$$

(c) Gradient accumulation computes the same gradient as a large batch, but it never stores activations for all B samples at once. Each mini-batch of size b fits in memory; gradients are accumulated in-place, and one optimizer step is taken after K mini-batches.

3. Consider a two-layer neural network defined as follows for a single training example $(x^{(i)}, y^{(i)})$:

$$z_1^{(i)} = W_1 x^{(i)} + b_1$$

$$a_1^{(i)} = \text{ReLU}(z_1^{(i)})$$

$$z_2^{(i)} = W_2 a_1^{(i)} + b_2$$

$$\hat{y}^{(i)} = \sigma(z_2^{(i)})$$

The binary cross-entropy loss is given by

$$L^{(i)} = [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

and the empirical risk over m samples is

$$J = \frac{-1}{m} \sum_{i=1}^m L^{(i)}.$$

Here, $x^{(i)}$ represents a single input example, and is of shape $\mathbb{R}^{D_x \times 1}$. $y^{(i)} \in \mathbb{R}$ is single output label and is a scalar. There are m samples in the dataset. The hidden layer z_1 has D_{a1} neurons.

- (a) What are the shapes of W_1, b_1, W_2, b_2 for a single example? If the network is vectorized over m examples, what are the shapes of the parameters? What are the shapes of X and Y after vectorization?

Solution: For a single example:

$$W_1 \in \mathbb{R}^{D_{a1} \times D_x}, \quad b_1 \in \mathbb{R}^{D_{a1} \times 1}$$

$$W_2 \in \mathbb{R}^{1 \times D_{a1}}, \quad b_2 \in \mathbb{R}^{1 \times 1}$$

After vectorization across m examples:

$$X \in \mathbb{R}^{D_x \times m}, \quad Y \in \mathbb{R}^{1 \times m}$$

The shapes of W_1, b_1, W_2, b_2 remain unchanged.

- (b) Compute $\frac{\partial J}{\partial \hat{y}^{(i)}}$. Refer to this quantity as $\delta_1^{(i)}$. What is $\frac{\partial J}{\partial \hat{y}}$?

Solution:

$$\delta_1^{(i)} = \frac{\partial L^{(i)}}{\partial \hat{y}^{(i)}} = \left(\frac{y^{(i)}}{\hat{y}^{(i)}} - \frac{1 - y^{(i)}}{1 - \hat{y}^{(i)}} \right)$$

$$\frac{\partial J}{\partial \hat{y}} = \frac{-1}{m} \sum_{i=1}^m \delta_1^{(i)}$$

- (c) Compute $\frac{\partial \hat{y}^{(i)}}{\partial z_2}$. Refer to this as $\delta_2^{(i)}$.

Solution: Since $\hat{y}^{(i)} = \sigma(z_2^{(i)})$,

$$\delta_2^{(i)} = \sigma(z_2)(1 - \sigma(z_2))$$

- (d) Compute $\frac{\partial z_2}{\partial a_1}$. Refer to this as $\delta_3^{(i)}$.

Solution:

$$\delta_3^{(i)} = W_2$$

- (e) Compute $\frac{\partial a_1}{\partial z_1}$. Refer to this as $\delta_4^{(i)}$.

Solution: For ReLU,

$$\delta_4^{(i)} = \begin{cases} 1 & \text{if } z_1^{(i)} > 0 \\ 0 & \text{if } z_1^{(i)} \leq 0 \end{cases}$$

- (f) Compute $\frac{\partial z_1}{\partial W_1}$. Refer to this as $\delta_5^{(i)}$.

Solution:

$$\delta_5^{(i)} = (x^{(i)})^T$$

- (g) Compute $\frac{\partial J}{\partial W_1}$. Carefully indicate the shapes.

Solution: Using the chain rule,

$$\frac{\partial J}{\partial W_1} = \frac{-1}{m} \sum_{i=1}^m \delta_1^{(i)} \cdot \delta_2^{(i)} \cdot \left(\delta_3^{(i)} \odot \delta_4^{(i)} \right) \cdot \delta_5^{(i)}$$

where $\odot$ denotes element-wise multiplication.

The resulting gradient has shape:

$$\frac{\partial J}{\partial W_1} \in \mathbb{R}^{D_{a1} \times D_x}.$$

4. Implement the Adam optimizer from scratch and compare it with vanilla Stochastic Gradient Descent (SGD) on a non-convex optimization problem.

Problem Statement

Consider the following non-convex function (Rosenbrock function):

$$f(x, y) = x^2 + 100(y - x^2)^2$$

Tasks

1. Implement the following optimization algorithms from scratch (do not use built-in optimizers from PyTorch/TensorFlow):

- Vanilla Gradient Descent (SGD)
 - Adam optimizer
2. Use the same learning rate 10^{-3} initialization for both methods.
 3. Optimize the function starting from the initial point:
$$(x_0, y_0) = (-1.5, 1.5)$$
 4. Run both optimizers for 5000 iterations.
 5. Plot the following:
 - Function value vs. iterations
 - Optimization trajectory on a 2D contour plot of the Rosenbrock function
 6. Compare the two methods in terms of:
 - Convergence speed
 - Stability of updates
 - Sensitivity to learning rate

$$\eta \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}\}$$

Solution:

5. Theory: Spectral Convergence of Optimization Methods

Let

$$f(w) = \frac{1}{2} w^T A w$$

where $A \in \mathbb{R}^{d \times d}$ is symmetric positive definite with eigenvalues

$$\lambda_1 \leq \dots \leq \lambda_d$$

Let the spectral decomposition of A be

$$A = Q \Lambda Q^T$$

where $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_d)$.

(a) Show that Gradient Descent with optimal fixed step size

$$\eta^* = \frac{2}{\lambda_1 + \lambda_d}$$

has convergence rate

$$\|w^{(k)} - w^*\|_2 \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^k \|w^{(0)} - w^*\|_2$$

where

$$\kappa = \frac{\lambda_d}{\lambda_1}$$

Hint:

- Compute $\nabla f(w)$ and write the Gradient Descent update.
- Express the error iteration using $e^{(k)} = w^{(k)} - w^*$.
- Use the spectral decomposition $A = Q\Lambda Q^T$ to analyze the behavior along eigen-directions.

(b) Explain mathematically why (using spectral arguments):

- Newton's method converges in one step.
- Conjugate Gradient converges in at most d steps.
- AdaGrad cannot eliminate dependence on κ completely.

Hint:

- Compute the gradient and Hessian of $f(w)$.
- Write the Newton update and simplify using properties of A .
- Consider how optimization methods behave along the eigenvectors of A .
- Recall that Conjugate Gradient constructs A -conjugate directions.

Solution:

(a) For

$$f(w) = \frac{1}{2}w^T Aw$$

the gradient is

$$\nabla f(w) = Aw$$

Thus the Gradient Descent update is

$$w^{(k+1)} = w^{(k)} - \eta Aw^{(k)}$$

Let the error be

$$e^{(k)} = w^{(k)} - w^*$$

Since the minimizer satisfies

$$\nabla f(w^*) = Aw^* = 0$$

we obtain

$$w^* = 0$$

Thus the error iteration becomes

$$e^{(k+1)} = (I - \eta A)e^{(k)}$$

Using the spectral decomposition

$$A = Q\Lambda Q^T$$

we obtain

$$e^{(k)} = Q(I - \eta\Lambda)^k Q^T e^{(0)}$$

Hence each eigen-component evolves as

$$z_i^{(k+1)} = (1 - \eta\lambda_i)z_i^{(k)}$$

The contraction factor is

$$\rho = \max_i |1 - \eta\lambda_i|$$

Minimizing this quantity gives

$$\eta^* = \frac{2}{\lambda_1 + \lambda_d}$$

which yields

$$\rho = \frac{\kappa - 1}{\kappa + 1}$$

where

$$\kappa = \frac{\lambda_d}{\lambda_1}$$

Therefore

$$\|e^{(k)}\|_2 \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^k \|e^{(0)}\|_2$$

(b) **Newton's Method**

For the quadratic function

$$\nabla f(w) = Aw$$

and

$$\nabla^2 f(w) = A$$

The Newton update is

$$w^{(k+1)} = w^{(k)} - (\nabla^2 f(w^{(k)}))^{-1} \nabla f(w^{(k)})$$

Substituting gives

$$w^{(k+1)} = w^{(k)} - A^{-1}Aw^{(k)}$$

which simplifies to

$$w^{(k+1)} = 0 = w^*$$

Thus all spectral components are eliminated in a single step.

—

Conjugate Gradient

Conjugate Gradient constructs search directions that are A -conjugate:

$$p_i^T A p_j = 0 \quad (i \neq j)$$

These directions span the Krylov subspace

$$\mathcal{K}_k(A, r_0) = \text{span}\{r_0, Ar_0, \dots, A^{k-1}r_0\}$$

In the eigenbasis of A , CG removes error components corresponding to eigenvalues through a sequence of polynomials in A .

Since a polynomial of degree d can cancel all d eigenvalues, the exact solution is obtained in at most d iterations.

—

AdaGrad

AdaGrad performs the update

$$w_{t+1} = w_t - \eta D_t^{-1/2} g_t$$

where D_t accumulates squared gradients and is diagonal.

This produces coordinate-wise scaling but does not align with the eigenvectors of A when A is not diagonal.

Therefore the eigenvalue spectrum of A is not fully corrected, and convergence still depends on the condition number

$$\kappa = \frac{\lambda_d}{\lambda_1}$$

6. Programming + Analysis: RMSProp vs Adam

Consider the non-convex function:

$$f(w_1, w_2) = \frac{1}{2}(w_1^2 + 1000w_2^2) + 5 \sin(w_1)$$

- (a) Implement RMSProp and Adam from scratch.
- (b) Initialize at

$$(w_1, w_2) = (2, 2)$$

Run for 200 iterations.

- (c) Report:
 - Final parameter values
 - Final loss
 - Plot loss vs iterations (log scale)

Solution:

(a) Step 1: Compute Gradients

First derive analytical gradients:

$$\frac{\partial f}{\partial w_1} = w_1 + 5 \cos(w_1)$$

$$\frac{\partial f}{\partial w_2} = 1000w_2$$

These gradients must be implemented explicitly in code.

Step 2: Implementation Strategy

- Define the objective function and gradient function separately.
 - Initialize parameters $(w_1, w_2) = (2, 2)$.
 - Use separate implementations for RMSProp and Adam.
 - Maintain history lists to store:
 - Parameter values
 - Loss values per iteration
 - Use small ϵ (e.g., 10^{-8}) to ensure numerical stability.
-

Step 3: RMSProp Update Rule

Maintain running average of squared gradients:

$$v_t = \beta v_{t-1} + (1 - \beta)g_t^2$$

Parameter update:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t} + \epsilon} g_t$$

Typical hyperparameters:

- $\beta = 0.9$
- $\eta = 0.001$ (or tuned appropriately)

RMSProp rescales updates coordinate-wise, helping with the large curvature caused by the $1000w_2^2$ term.

—

Step 4: Adam Update Rule

Maintain first and second moment estimates:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Update rule:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Typical hyperparameters:

- $\beta_1 = 0.9$
- $\beta_2 = 0.999$
- $\eta = 0.001$

Adam combines momentum and adaptive scaling, typically leading to faster convergence.

—

(b) Execution

- Run both optimizers for 200 iterations.
- Record loss at each iteration.
- Track parameter trajectory to observe convergence behavior.
- Expect rapid stabilization along w_2 direction due to large curvature.

—

(c) Reporting and Analysis

- Print final values of (w_1, w_2) .

- Print final loss.
- Plot loss vs iterations using logarithmic scale on y-axis.
- Compare:
 - Speed of convergence
 - Stability of updates
 - Sensitivity to learning rate
- Discuss how adaptive learning rates help mitigate ill-conditioning caused by the $1000w_2^2$ term.

7. (Solve by hand) You are training a 3-layer neural network for a regression task. The network consists of an input layer (size 3), two hidden layers (size 3 each), and a linear output layer (size 1).

Network Architecture:

- Layer 1 (Hidden): 3 nodes. Activation: ReLU.
- Layer 2 (Hidden): 3 nodes. Activation: Sigmoid.
- Layer 3 (Output): 1 node. Activation: Linear (None).
- Loss Function: Squared Error, defined as $L = (\hat{y} - y)^2$

Initial Values: Input Vector (X): $\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$

Target Label (y): 1

Learning Rate (η): 0.01

Initial Weights & Biases: (Note: Biases B_1, B_2, B_3 are all initialized to zero vectors).

$$W_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 2 \end{bmatrix}$$

$$W_2 = \begin{bmatrix} -1 & 0 & 0.5 \\ 1 & 0 & -0.5 \\ 2 & 0 & -1 \end{bmatrix}$$

$$W_3 = \begin{bmatrix} 2 & 2 & 2 \end{bmatrix}$$

Your Task:

1. Perform a forward pass to compute the network's prediction ($\hat{y}$) and the Loss (L).
2. Perform a backward pass to compute the gradients of the loss with respect to all weights and biases.

3. Execute one step of Gradient Descent to calculate the updated weights (W_1, W_2, W_3) and biases (B_1, B_2, B_3)

Solution:

1. The Forward Pass

We compute the values of all nodes from input to output.

$$\text{Layer 1 (ReLU): } Z_1 = W_1 X + B_1 = \begin{bmatrix} 1(1) \\ -1(1) \\ 2(1) \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \\ 2 \end{bmatrix}$$

$$A_1 = \max(Z_1, 0) = \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}$$

$$\text{Layer 2 (Sigmoid): } Z_2 = W_2 A_1 + B_2 = \begin{bmatrix} -1(1) + 0 + 0.5(2) \\ 1(1) + 0 - 0.5(2) \\ 2(1) + 0 - 1(2) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$A_2 = \frac{1}{1 + \exp(-Z_2)} = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \end{bmatrix} \quad (\text{Since } \exp(0) = 1)$$

$$\text{Layer 3 (Linear Output): } \hat{y} = Z_3 = W_3 A_2 + B_3 = 2(0.5) + 2(0.5) + 2(0.5) + 0 = 3$$

$$\text{Loss (L): } L = (\hat{y} - y)^2 = (3 - 1)^2 = 4$$

2. The Backward Pass

We compute the gradients using the forward pass values and the chain rule, starting from the last layer.

Output Layer Gradients:

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y) = 2(3 - 1) = 4$$

$$\frac{\partial L}{\partial B_3} = \frac{\partial L}{\partial \hat{y}} \cdot 1 = 4$$

$$\frac{\partial L}{\partial W_3} = \frac{\partial L}{\partial \hat{y}} \cdot A_2^T = 4 \times \begin{bmatrix} 0.5 & 0.5 & 0.5 \end{bmatrix} = \begin{bmatrix} 2 & 2 & 2 \end{bmatrix}$$

Hidden Layer 2 Gradients: Let $\delta_2 = \frac{\partial L}{\partial Z_2}$.

The derivative of Sigmoid is $A_2 \cdot (1 - A_2)$.

For 0.5, this is $0.5 \times 0.5 = 0.25$.

$$\delta_2 = (W_3^T \cdot \frac{\partial L}{\partial \hat{y}}) \odot \sigma'(Z_2) = \left(\begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix} \times 4 \right) \odot \begin{bmatrix} 0.25 \\ 0.25 \\ 0.25 \end{bmatrix} = \begin{bmatrix} 8 \\ 8 \\ 8 \end{bmatrix} \odot \begin{bmatrix} 0.25 \\ 0.25 \\ 0.25 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix}$$

$$\frac{\partial L}{\partial B_2} = \delta_2 = \begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix} \quad \frac{\partial L}{\partial W_2} = \delta_2 \cdot A_1^T = \begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix} \begin{bmatrix} 1 & 0 & 2 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 4 \\ 2 & 0 & 4 \\ 2 & 0 & 4 \end{bmatrix}$$

Hidden Layer 1 Gradients: Let $\delta_1 = \frac{\partial L}{\partial Z_1}$.

The derivative of ReLU is 1 if $Z_1 > 0$ else 0. For $Z_1 = [1, -1, 2]^T$, the derivative is $[1, 0, 1]^T$.

$$\delta_1 = (W_2^T \cdot \delta_2) \odot \text{ReLU}'(Z_1) W_2^T \cdot \delta_2 = \begin{bmatrix} -1 & 1 & 2 \\ 0 & 0 & 0 \\ 0.5 & -0.5 & -1 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix} = \begin{bmatrix} -2 + 2 + 4 \\ 0 \\ 1 - 1 - 2 \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \\ -2 \end{bmatrix}$$

$$\delta_1 = \begin{bmatrix} 4 \\ 0 \\ -2 \end{bmatrix} \odot \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \\ -2 \end{bmatrix}$$

$$\frac{\partial L}{\partial B_1} = \delta_1 = \begin{bmatrix} 4 \\ 0 \\ -2 \end{bmatrix} \quad \frac{\partial L}{\partial W_1} = \delta_1 \cdot X^T = \begin{bmatrix} 4 \\ 0 \\ -2 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 4 & 4 & 4 \\ 0 & 0 & 0 \\ -2 & -2 & -2 \end{bmatrix}$$

3. Gradient Descent Optimization Using the update rule

$W_{n+1} = W_n - \eta \nabla L(w_n)$ with $\eta = 0.01$.

Layer 3 Updates: $W_3^{new} = \begin{bmatrix} 2 & 2 & 2 \end{bmatrix} - 0.01 \begin{bmatrix} 2 & 2 & 2 \end{bmatrix} = \begin{bmatrix} 1.98 & 1.98 & 1.98 \end{bmatrix}$

$B_3^{new} = 0 - 0.01(4) = -0.04$

$$\text{Layer 2 Updates: } W_2^{new} = \begin{bmatrix} -1 & 0 & 0.5 \\ 1 & 0 & -0.5 \\ 2 & 0 & -1 \end{bmatrix} - 0.01 \begin{bmatrix} 2 & 0 & 4 \\ 2 & 0 & 4 \\ 2 & 0 & 4 \end{bmatrix} = \begin{bmatrix} -1.02 & 0 & 0.46 \\ 0.98 & 0 & -0.54 \\ 1.98 & 0 & -1.04 \end{bmatrix}$$

$$B_2^{new} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - 0.01 \begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix} = \begin{bmatrix} -0.02 \\ -0.02 \\ -0.02 \end{bmatrix}$$

$$\text{Layer 1 Updates: } W_1^{new} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 2 \end{bmatrix} - 0.01 \begin{bmatrix} 4 & 4 & 4 \\ 0 & 0 & 0 \\ -2 & -2 & -2 \end{bmatrix} = \begin{bmatrix} 0.96 & -0.04 & -0.04 \\ 0 & -1 & 0 \\ 0.02 & 0.02 & 2.02 \end{bmatrix}$$

$$B_1^{new} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - 0.01 \begin{bmatrix} 4 \\ 0 \\ -2 \end{bmatrix} = \begin{bmatrix} -0.04 \\ 0 \\ 0.02 \end{bmatrix}$$

8. (Coding and Analysis) Empirically validate the need for specific weight initialization strategies.

Task: Build a 2-layer Multilayer Perceptron (MLP) without using high-level framework layers (e.g., use NumPy matrix multiplications). Pass a batch of normally distributed inputs through the network. Record and plot the variance of the activations (plot the histogram of each neurons activation value) at each layer using four different initialization schemes: Zero initialization, standard random normal initialization, Xavier initialization for Sigmoid activations ($\mathcal{N}(0, \frac{2}{n_{in}+n_{out}})$), and Kaiming He initialization for ReLU activations ($\mathcal{N}(0, \frac{2}{n_{in}})$)

Network configuration

- Batch size: 128
- Input size: 3
- Hidden layer sizes (layers 1, 2): 5

- Output layer size: 5 (no activation; just for completeness as we are not training here)

Initialization Type	Network 1 (Hidden Activation)	Network 2 (Hidden Activation)
Zero	ReLU	Sigmoid
Standard Normal	ReLU	Sigmoid
Xavier (Glorot)	Sigmoid	-
Kaiming (He)	ReLU	-

Table 1: Neural networks to build for this exercise

Total 6 networks (See Table 1). Visualise the activation distribution as a histogram for each neuron in the entire network and provide your justification on the effects of weight initialisation. It will be 128 (batch size) values per neuron, and make a histogram of it for each hidden neuron.

Solution: -

9. Numerical: Two Steps of Adam Optimizer

Consider minimizing the scalar function:

$$f(w) = \frac{1}{2}w^2$$

Consider:

- w_t : parameter at iteration t
- g_t : gradient at iteration t
- β_1 : exponential decay rate for first moment
- β_2 : exponential decay rate for second moment
- η : learning rate
- ϵ : small constant for numerical stability

Let:

$$w_0 = 2, \quad \eta = 0.1, \quad \beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 0$$

- Compute w_1 weight after 1st iteration.
- Compute w_2 weight after 2nd iteration.

Solution:

m_t : first moment estimate (momentum term)

v_t : second moment estimate (squared gradient average)

Step 1

$$g_1 = w_0 = 2$$

$$m_1 = 0.1(2) = 0.2$$

$$v_1 = 0.001(4) = 0.004$$

Now directly update using m_1 and v_1 (no bias correction):

$$w_1 = 2 - \frac{0.1}{\sqrt{0.004}}(0.2)$$

$$\sqrt{0.004} \approx 0.063245$$

$$w_1 = 2 - \frac{0.1}{0.063245}(0.2)$$

$$= 2 - (1.5811)(0.2)$$

$$= 2 - 0.3162$$

$$w_1 \approx 1.6838$$

—

Step 2

$$g_2 = w_1 = 1.6838$$

$$m_2 = 0.9(0.2) + 0.1(1.6838)$$

$$= 0.18 + 0.16838 = 0.34838$$

$$v_2 = 0.999(0.004) + 0.001(1.6838^2)$$

$$1.6838^2 \approx 2.835$$

$$v_2 = 0.003996 + 0.002835 = 0.006831$$

Update:

$$w_2 = 1.6838 - \frac{0.1}{\sqrt{0.006831}}(0.34838)$$

$$\sqrt{0.006831} \approx 0.08265$$

$$w_2 = 1.6838 - \frac{0.1}{0.08265}(0.34838)$$

$$= 1.6838 - (1.210)(0.34838)$$

$$= 1.6838 - 0.4214$$

$$w_2 \approx 1.2624$$

10. Coding: RMSProp with ReLU Activation

Consider a single neuron model with ReLU activation:

$$z = wx$$

$$a = \text{ReLU}(z) = \max(0, z)$$

Loss function:

$$L = \frac{1}{2}(a - y)^2$$

RMSProp update:

$$v_t = \beta v_{t-1} + (1 - \beta)g_t^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t} + \epsilon} g_t$$

Where:

- x : input
- y : target output
- w : weight parameter
- $g_t = \frac{\partial L}{\partial w}$: gradient
- β : decay rate
- η : learning rate
- ϵ : numerical stability term

Task: Code RMSProp from scratch

Take initialize: $w = 1.5$

Input: $x = 2$

Target: $y = 1$

$\eta = 0.01$, $\beta = 0.9$

Run for 20 iterations and Plot loss vs iteration

Solution:

11. **(Theoretical Question)** A recommendation system predicts whether a user will **like** a movie ($y = 1$) or **not like** it ($y = 0$) using two features:

- $x_1 \in \{0, 1\}$: whether the movie is **fiction** (1 = fiction, 0 = non-fiction) — a **dense** feature, present for every movie. The user generally likes fiction and dislikes non-fiction.
- $x_2 \in \{0, 1\}$: whether the movie is **directed by Director Y** — a **sparse** feature, true for very few movies. Whenever Director Y directs, the user likes the movie regardless of genre.

The model is logistic regression $\hat{y} = \sigma(w_1x_1 + w_2x_2)$ with cross-entropy loss. The per-sample gradient is:

$$g_i^{(t)} = (\hat{y}^{(t)} - y^{(t)}) \cdot x_i^{(t)}$$

SGD is run for one epoch over the following 6 movies, with $\hat{y} \approx 0.5$ throughout and $\eta = 0.5$:

Sample	x_1 (fiction)	x_2 (Dir. Y)	y (liked)	Reason
1	1	0	1	fiction $\rightarrow$ liked
2	0	0	0	non-fiction $\rightarrow$ not liked
3	1	0	1	fiction $\rightarrow$ liked
4	0	0	0	non-fiction $\rightarrow$ not liked
5	1	0	1	fiction $\rightarrow$ liked
6	0	1	1	non-fiction but Dir. Y $\rightarrow$ liked

Observe: x_1 follows a clean pattern across all 6 samples. $x_2 = 1$ only in sample 6 — a non-fiction movie that the user likes *only* because of Director Y. This is the strongest and cleanest signal in the data, yet w_2 receives a gradient update only once in the entire epoch.

- (a) Compute the net weight update for each weight over the full epoch:

$$\Delta w_i = -\eta \sum_{t=1}^6 g_i^{(t)}$$

Show your working sample by sample for both w_1 and w_2 .

- (b) You should observe that w_2 receives far fewer non-zero gradient updates than w_1 across the epoch. Explain precisely why sparsity of x_2 causes learning signal scarcity of w_2 , and why a uniform learning rate η cannot compensate for this imbalance even when x_2 is the stronger predictor.
- (c) AdaGrad maintains a per-parameter sum of squared gradients $G_i = \sum_{t=1}^6 (g_i^{(t)})^2$ and replaces the fixed learning rate with:

$$\eta_i^{\text{eff}} = \frac{\eta}{\sqrt{G_i + \epsilon}}$$

Using your gradients from part (a), compute G_1 , G_2 , and the ratio $\eta_2^{\text{eff}}/\eta_1^{\text{eff}}$ (use $\epsilon = 10^{-8}$). Explain how this ratio reflects AdaGrad's response to gradient starvation.

Solution:

Part (a)

With $\hat{y} \approx 0.5$, the error term $(\hat{y} - y)$ is $+0.5$ when $y = 0$ and -0.5 when $y = 1$.

Gradients for w_1 :

$$\begin{aligned}
g_1^{(1)} &= (-0.5)(1) = -0.50 && \text{(fiction, liked)} \\
g_1^{(2)} &= (+0.5)(0) = 0.00 && \text{(non-fiction, not liked)} \\
g_1^{(3)} &= (-0.5)(1) = -0.50 && \text{(fiction, liked)} \\
g_1^{(4)} &= (+0.5)(0) = 0.00 && \text{(non-fiction, not liked)} \\
g_1^{(5)} &= (-0.5)(1) = -0.50 && \text{(fiction, liked)} \\
g_1^{(6)} &= (-0.5)(0) = 0.00 && \text{(non-fiction, Dir. Y, liked)}
\end{aligned}$$

$$\sum_{t=1}^6 g_1^{(t)} = -0.50 + 0 - 0.50 + 0 - 0.50 + 0 = -1.50$$

$$\Delta w_1 = -0.5 \times (-1.50) = +0.75$$

Gradients for w_2 :

$$\begin{aligned}
g_2^{(1)} &= (-0.5)(0) = 0 \\
g_2^{(2)} &= (+0.5)(0) = 0 \\
g_2^{(3)} &= (-0.5)(0) = 0 \\
g_2^{(4)} &= (+0.5)(0) = 0 \\
g_2^{(5)} &= (-0.5)(0) = 0 \\
g_2^{(6)} &= (-0.5)(1) = -0.5
\end{aligned}$$

$$\sum_{t=1}^6 g_2^{(t)} = -0.5$$

$$\Delta w_2 = -0.5 \times (-0.5) = +0.25$$

Part (b)

w_1 receives a non-zero gradient at **3 out of 6 samples** (samples 1, 3, 5 where $x_1 = 1$).
 w_2 receives a non-zero gradient at **only 1 out of 6 samples** (sample 6). This follows directly from the gradient structure:

$$g_i^{(t)} = (\hat{y}^{(t)} - y^{(t)}) \cdot x_i^{(t)} = 0 \quad \text{whenever } x_i^{(t)} = 0$$

The prediction error on samples 1–5 is completely invisible to w_2 because $x_2 = 0$ zeroes out the gradient regardless of how wrong the model is. SGD applies the same $\eta = 0.5$ to both weights with no awareness of update frequency. The result: $\Delta w_1 = +0.75$ while $\Delta w_2 = +0.25$, meaning w_1 moves **three times further** in one epoch. Director Y's weight is gradient-starved not because the signal is weak, but because it arrives only once — and a uniform learning rate has no mechanism to detect or correct this.

Part (c)

Computing $G_i = \sum_{t=1}^6 (g_i^{(t)})^2$:

$$\begin{aligned} G_1 &= (-0.50)^2 + (0)^2 + (-0.50)^2 + (0)^2 + (-0.50)^2 + (0)^2 \\ &= 0.25 + 0.25 + 0.25 = 0.75 \end{aligned}$$

$$G_2 = 0 + 0 + 0 + 0 + 0 + (-0.5)^2 = 0.25$$

Effective learning rates ($\epsilon \approx 0$):

$$\eta_1^{\text{eff}} = \frac{0.5}{\sqrt{0.75}} = \frac{0.5}{0.866} \approx 0.577$$

$$\eta_2^{\text{eff}} = \frac{0.5}{\sqrt{0.25}} = \frac{0.5}{0.5} = 1.0$$

$$\frac{\eta_2^{\text{eff}}}{\eta_1^{\text{eff}}} = \frac{1.0}{0.577} \approx 1.73$$

AdaGrad assigns w_2 an effective learning rate **1.73 times larger** than w_1 . G_2 is small because w_2 has only ever seen one non-zero gradient — so AdaGrad keeps its step size large, ensuring that sample 6 produces a substantially larger update than SGD would allow. G_1 has accumulated three non-zero squared gradients, reducing η_1^{eff} to prevent overshooting on the already well-trained fiction signal. AdaGrad thus **automatically compensates for gradient starvation**: a weight that has been starved of updates earns a larger effective step precisely when its signal finally arrives.

12. Coding: Optimization in a Non-Convex Loss

Consider the non-convex function

$$L(x) = x^4 - 4x^2 + x.$$

1. Plot $L(x)$ for $x \in [-3, 3]$ and identify the number of local minima.
2. Implement Gradient Descent and Adam from scratch.
3. Run both optimizers for 300 iterations from:

$$x_0 = -1.5 \quad \text{and} \quad x_0 = +1.5.$$

4. Plot the loss curve and overlay the optimization trajectories.
5. Report the final converged values. Briefly explain:
 - Why different initializations converge to different minima.
 - Why fixing random seeds is important for reproducibility.
 - Why trying multiple seeds can help discover better minima in non-convex problems.

Solution:

13. Effect of Initialization on Gradient Flow

Consider a deep neural network with 15 hidden layers using ReLU activation. All weights are initialized from $\mathcal{N}(0, 0.01^2)$ (very small variance).

Which of the following is MOST likely to happen during the first few training epochs? Give a detailed explanation for your choice.

- (A) Activations explode as depth increases.
- (B) Activations shrink toward zero as depth increases.
- (C) Gradients become exactly zero for all neurons.
- (D) The network immediately reaches optimal performance

Solution:

Correct Answer: **(B)**

If weights are initialized with very small variance, each layer scales the signal down slightly. Across many layers, this leads to shrinking activations and shrinking gradients (even without classical vanishing gradient caused by sigmoid/tanh saturation).

This demonstrates the importance of variance-preserving initialization such as Xavier initialization.

14. Programming : Comparing SGD and Momentum

Consider minimizing the function:

$$f(w_1, w_2) = \frac{1}{2}(10w_1^2 + w_2^2)$$

(a) Implement from scratch:

- Vanilla Gradient Descent
- Gradient Descent with Momentum

Use the update rules:

SGD:

$$w_{t+1} = w_t - \eta g_t$$

Momentum:

$$v_t = \beta v_{t-1} + g_t$$

$$w_{t+1} = w_t - \eta v_t$$

(b) Initialize:

$$(w_1, w_2) = (2, 2)$$

Use:

$$\eta = 0.1, \quad \beta = 0.9$$

Run for 100 iterations.

(c) Report:

- Final parameter values
- Final loss
- Plot loss vs iterations (log scale)
- Plot trajectory in (w_1, w_2) space

(d) Explain why Momentum converges faster along the shallow curvature direction compared to vanilla SGD. Relate your explanation to the eigenvalues of the Hessian.

Solution:

Gradient Computation

$$\frac{\partial f}{\partial w_1} = 10w_1$$

$$\frac{\partial f}{\partial w_2} = w_2$$

The Hessian matrix is:

$$H = \begin{bmatrix} 10 & 0 \\ 0 & 1 \end{bmatrix}$$

Its eigenvalues are $\lambda_1 = 1$, $\lambda_2 = 10$.

This creates an ill-conditioned optimization landscape with condition number:

$$\kappa = \frac{10}{1} = 10$$

Vanilla SGD oscillates along the steep w_1 direction while progressing slowly along the shallow w_2 direction.

Momentum accumulates gradients in consistent directions, effectively accelerating convergence along the shallow curvature direction while damping oscillations along the steep direction.

Thus momentum reduces zig-zag behavior and improves convergence speed in ill-conditioned problems.

15. Programming: Gradient Descent vs Newton's Method

Consider the function

$$f(w_1, w_2) = 3w_1^2 + 2w_1w_2 + 2w_2^2$$

The loss value at iteration t is defined as

$$L_t = f(w_1^{(t)}, w_2^{(t)})$$

where $w_1^{(t)}$ and $w_2^{(t)}$ denote the parameter values at iteration t .

The Hessian of this function is constant and positive definite, therefore the function is convex and has a unique global minimum.

Perform the following tasks.

1. Derive the gradient vector $\nabla f(w)$ and the Hessian matrix $\nabla^2 f(w)$.
2. Implement the following optimization algorithms. Use PyTorch, TensorFlow, or built-in optimizer libraries.
 - Gradient Descent
 - Newton's Method
3. Initialize the parameters as

$$w = \begin{bmatrix} 4 \\ 4 \end{bmatrix}$$

4. Use learning rate

$$\eta = 0.1$$

for Gradient Descent.

5. Use the following Newton update rule (without a learning rate):

$$w_{t+1} = w_t - (\nabla^2 f)^{-1} \nabla f(w_t)$$

6. Run Gradient Descent for **50 iterations** and Newton's Method for **5 iterations**.

7. Plot

- Loss L_t versus iteration number
- Optimization trajectory in the (w_1, w_2) plane

8. Briefly compare the convergence behavior of the two methods.

Solution:

1. Gradient Derivation

$$f(w_1, w_2) = 3w_1^2 + 2w_1w_2 + 2w_2^2$$

Partial derivatives:

$$\frac{\partial f}{\partial w_1} = 6w_1 + 2w_2$$

$$\frac{\partial f}{\partial w_2} = 2w_1 + 4w_2$$

Thus

$$\nabla f(w) = \begin{bmatrix} 6w_1 + 2w_2 \\ 2w_1 + 4w_2 \end{bmatrix}$$

2. Hessian Matrix

$$\nabla^2 f = \begin{bmatrix} 6 & 2 \\ 2 & 4 \end{bmatrix}$$

Since

$$\det(\nabla^2 f) = 24 - 4 = 20 > 0$$

and the leading principal minor is positive, the Hessian is positive definite. Therefore the function is convex.

3. Gradient Descent Update

$$w_{t+1} = w_t - \eta \nabla f(w_t)$$

Algorithm outline:

- Initialize

$$w = \begin{bmatrix} 4 \\ 4 \end{bmatrix}$$

- For $t = 1$ to 50:
 - Compute gradient

$$g = \nabla f(w)$$

- Update

$$w = w - \eta g$$

- Compute loss

$$L_t = f(w)$$

4. Newton's Method Update

$$w_{t+1} = w_t - (\nabla^2 f)^{-1} \nabla f(w_t)$$

First compute inverse Hessian.

$$(\nabla^2 f)^{-1} = \frac{1}{20} \begin{bmatrix} 4 & -2 \\ -2 & 6 \end{bmatrix}$$

Algorithm outline:

- Initialize

$$w = \begin{bmatrix} 4 \\ 4 \end{bmatrix}$$

- For $t = 1$ to 5:
 - Compute gradient

$$g = \nabla f(w)$$

– Update

$$w = w - (\nabla^2 f)^{-1}g$$

– Compute loss

$$L_t = f(w)$$

5. Expected Behaviour

- Gradient Descent converges gradually and requires many iterations depending on the learning rate.
- Newton's Method uses curvature information from the Hessian and therefore converges much faster.
- For quadratic functions, Newton's Method typically reaches the optimum in very few iterations.

16. Numerical: Adam Update in Two Dimensions

Consider the function

$$f(w_1, w_2) = w_1^2 + 4w_2^2$$

At iteration $t = 1$, the parameter vector is

$$w^{(0)} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

The Adam optimizer uses the following update rules:

$$g_t = \nabla f(w_t)$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$$

Bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Parameter update:

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Given:

$$\eta = 0.1, \quad \beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 10^{-8}$$

Assume

$$m_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad v_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Compute the updated parameter vector $w^{(1)}$ after one Adam update step.

Solution:

Step 1: Gradient

$$\nabla f = \begin{bmatrix} 2w_1 \\ 8w_2 \end{bmatrix}$$

At

$$w^{(0)} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$g_1 = \begin{bmatrix} 2 \\ 16 \end{bmatrix}$$

Step 2: First moment

$$m_1 = 0.9m_0 + 0.1g_1$$

$$m_1 = \begin{bmatrix} 0.2 \\ 1.6 \end{bmatrix}$$

Step 3: Second moment

$$v_1 = 0.999v_0 + 0.001g_1^2$$

$$g_1^2 = \begin{bmatrix} 4 \\ 256 \end{bmatrix}$$

$$v_1 = \begin{bmatrix} 0.004 \\ 0.256 \end{bmatrix}$$

Step 4: Bias correction

$$\hat{m}_1 = \frac{m_1}{1 - 0.9} = \begin{bmatrix} 2 \\ 16 \end{bmatrix}$$

$$\hat{v}_1 = \frac{v_1}{1 - 0.999} = \begin{bmatrix} 4 \\ 256 \end{bmatrix}$$

Step 5: Parameter update

$$w^{(1)} = w^{(0)} - 0.1 \begin{bmatrix} \frac{2}{\sqrt{4}} \\ \frac{16}{\sqrt{256}} \end{bmatrix}$$

$$= \begin{bmatrix} 1 \\ 2 \end{bmatrix} - 0.1 \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0.9 \\ 1.9 \end{bmatrix}$$

$$\boxed{w^{(1)} = \begin{bmatrix} 0.9 \\ 1.9 \end{bmatrix}}$$